

AdaptED

Software Tools, Model Selection and Technology Justification

An Intelligent Multiagent Adaptive Education Assistant
with an Outcome Based Education Mapping Agent

Technical design and justification note

Abstract

This document records the software tools selected for the AdaptED platform, states the justification for each selection against the alternatives considered, and sets out the reasoning behind routing all large language model traffic through the OpenRouter aggregation gateway rather than integrating vendor interfaces directly. It further documents the two models presently in service, Poolside Laguna S 2.1 for generation and NVIDIA Llama Nemotron Embed VL 1B v2 for retrieval embeddings, together with the engineering constraints that each imposes on the surrounding system. A risk register and a configuration reference are included so that the design can be reproduced and reviewed independently. The governing principle throughout is that a research prototype must remain reproducible, inexpensive to operate and free of dependence on any single vendor.

Contents

1. Purpose and scope

2. Software tool stack

2.1 Selection method

2.2 Selected tools and justification

3. Justification for the OpenRouter gateway

3.1 Consolidation of vendor integrations

3.2 Schema compatibility and a minimal abstraction layer

3.3 Model substitution as a configuration change

3.4 Availability of a zero cost tier

3.5 Resilience through provider failover

3.6 Unified credential across both request types

3.7 Reproducibility and audit

3.8 Limitations of the approach

4. Model selection

4.1 Generation model: Poolside Laguna S 2.1

4.2 Embedding model: NVIDIA Llama Nemotron Embed VL 1B v2

5. Deterministic offline execution mode

6. Risk register

7. Configuration reference

8. Validity of this document

References

3

3

3

3

6

6

6

6

6

6

6

6

6

6

7

7

8

9

9

10

10

10

1. Purpose and scope

AdaptED converts a teacher's raw course material into an adaptive learning experience through a supervisor orchestrated group of specialised agents, and separately assists faculty in validating the outcome mappings of an AIUB Computer Science course outline. Both functions depend on a stack of third party software and on external model providers. The purpose of this document is to make those dependencies explicit and defensible.

Four constraints governed every selection recorded here:

- **Operating cost.** The platform must be developable, testable and demonstrable at zero or near zero cost, without a commercial budget and without an international payment instrument.
- **Reproducibility.** Any result presented must be regenerable at a later date, including in the absence of a network connection or a valid credential.
- **Portability.** No component may bind the system to a single vendor, since model availability and pricing change on a weekly cycle.
- **Determinism where correctness is required.** The outcome based education compliance checks produce a verdict that a faculty member may act upon, and must therefore be rule driven and explainable rather than probabilistic.

2. Software tool stack

2.1 Selection method

Each layer of the system was assessed against the four constraints in Section 1, against the specific demands of the architecture, and against the alternatives in common use. The outcome is recorded in Table 1. Where a selection is justified at length elsewhere in this document, the table refers forward to the relevant section rather than repeating the argument.

2.2 Selected tools and justification

Table 1. Software tool stack with justification by layer.

Layer	Tool	Role in AdaptED	Justification
Language	Python 3.12+	Backend, agents and retrieval pipeline	The dominant language for machine learning and language model tooling. Every library selected below is natively available in Python, which avoids cross language bindings.
Environment and packaging	uv	Dependency resolution, virtual environment and task execution	Resolves and installs substantially faster than pip, and produces a single lockfile so that the build is reproducible on a reviewer's machine without version drift.
Agent orchestration	LangGraph	Compiles the agent pipeline for each run, providing conditional routing, bounded retries and savepoints	The adaptive loop is a stateful graph containing a branch between advancement and remediation, and a cycle that returns a student to reassessment. It is not a linear chain. LangGraph expresses cycles and conditional edges natively, whereas a sequential chain framework cannot represent the remediation loop without external control logic.
Web interface layer	FastAPI with Uvicorn	REST routers, token protected endpoints and the background task pool	Asynchronous by default, which suits agent tasks of long duration. Generates its own specification documents, and integrates natively with Pydantic so that transport schemas and agent output schemas are validated by a single set of models.

Layer	Tool	Role in AdaptED	Justification
User interface	Streamlit	Teacher and student interfaces, including the outcome mapping page	A user interface written entirely in Python removes the need for a separate JavaScript application and build toolchain. This is proportionate for a research prototype whose contribution is the agent architecture rather than the presentation layer.
Relational database	PostgreSQL	Users, courses, tasks, the message bus, the audit log and the student profile	Transactional integrity is a functional requirement, because a failed agent attempt must roll back only its own writes while preserving the surrounding run. PostgreSQL additionally hosts the vector store, so a single engine serves both relational and vector workloads.
Vector store	pgvector	Storage and similarity retrieval of embeddings	Holding vectors within the relational database allows a retrieval result to be joined directly to its chapter, topic and course records. This avoids operating and synchronising a second datastore such as Pinecone, Qdrant or Chroma, and removes an entire class of consistency failure.
Object mapping and migration	SQLAlchemy 2.0 with Alembic	Model layer and schema version control	Typed declarative mapping in the 2.0 style. Alembic provides reviewable and reversible schema migrations in place of unmanaged SQL, which matters once a populated vector column must be altered.
Validation and configuration	Pydantic and pydantic-settings	Agent output schemas, transport models and environment configuration	Every agent returns output validated against a declared schema. This validation is the mechanism by which an unconstrained model response becomes safe to pass to the next agent, as explained in Section 4.1.
Model gateway	OpenRouter, accessed through an OpenAI compatible client	Single endpoint for both generation and embedding requests	Justified in full in Section 3.
Generation model	poolside/laguna-s-2.1:free	Lessons, assessments, study plans, recommendations and summary prose	Justified in full in Section 4.1.
Embedding model	nvidia/llama-nemotron-embed-v1-1b-v2:free	Vectorisation of course material and of incoming queries	Justified in full in Section 4.2.
Offline execution mode	Deterministic mock provider	Executes the complete system without a credential or a network	Justified in full in Section 5.
Document processing	pypdf and python-docx	Reads uploaded textbooks and AIUB course outlines	python-docx exposes the table structure of a Word document. This is essential rather than convenient, because the course outcome mappings reside inside tables rather than in running text, and a plain text extractor discards the row and column relationships on which the validation rules depend.
Authentication	PyJWT and bcrypt	Teacher and student roles across token protected routers	A stateless token suits an architecture in which the interface and the interface layer are separately deployed. bcrypt is the established adaptive hashing function for stored credentials.
Testing	pytest	Unit and end to end suites, including a genuine AIUB outline processed in full	Fixtures and conditional skipping allow the database dependent tests to be omitted automatically when no database is configured, so that the suite remains executable in a bare environment.

Layer	Tool	Role in AdaptED	Justification
Static analysis	ruff	Linting and formatting	A single fast tool replacing flake8, isort and black, which keeps the codebase internally consistent and reduces review overhead.

3. Justification for the OpenRouter gateway

OpenRouter is an aggregation gateway. A single credential and a single request format provide access to models from a large number of vendors, with generation and embedding requests served from the same base address [1][2]. AdaptED uses it for both. Seven considerations supported this decision, and one significant limitation is recorded against them in Section 3.8.

3.1 Consolidation of vendor integrations

Direct integration would require a separate account, credential, billing relationship, client library and error vocabulary for each vendor whose model is used. The gateway reduces this to one credential, one account and one request format. For a project maintained by a single researcher, the administrative and engineering overhead of several vendor relationships cannot be justified against the benefit obtained.

3.2 Schema compatibility and a minimal abstraction layer

Because the gateway follows the established OpenAI request schema, the provider module requires only a base address, a credential and a model identifier. No vendor specific client library is imported anywhere in the codebase. The abstraction layer therefore remains small enough to be reviewed in full, and the deterministic mock provider described in Section 5 becomes directly interchangeable with the live one.

3.3 Model substitution as a configuration change

The identity of the model resides in an environment variable rather than in application code. Comparing two generation models for an evaluation therefore requires a single line of configuration to be edited rather than a client to be rewritten. This is of direct benefit to a study that must report comparative results across models.

3.4 Availability of a zero cost tier

Both models presently in service are provided at no charge [3][5]. The platform can accordingly be developed, tested and demonstrated without a payment instrument. Given that access to international card payment is a practical obstacle for many students in Bangladesh, this is a material rather than an incidental advantage.

3.5 Resilience through provider failover

A single submission to the adaptive loop initiates a sequence of model calls covering grading, performance analysis, recommendation, plan adaptation, reteaching and reassessment. An outage at any point in that sequence fails the entire run. The gateway monitors provider health and transfers requests to the next available provider, and the same routing controls apply to an embedding request as to a generation request [2].

3.6 Unified credential across both request types

The gateway exposes a dedicated embeddings endpoint alongside the generation endpoint at the same base address and under the same credential [1]. The indexing path and the generation path therefore share one credential, one routing policy and one error handling path, rather than the platform holding one credential for a generation vendor and a second for an embedding vendor.

3.7 Reproducibility and audit

The exact model identifier is fixed in configuration and recorded against every persisted task and every message hop. Any run reported in an evaluation can accordingly be traced to the model that produced it, which is a precondition for a defensible experimental account.

3.8 Limitations of the approach

The argument above should be stated alongside its costs. The gateway introduces an additional network hop and therefore additional latency. It introduces a dependency whose own availability becomes material to the system. It offers less contractual control than a direct vendor agreement, and its zero cost endpoints are rate limited and subject to withdrawal, as documented in Section 6. For a production deployment supported by an

institutional budget, direct integration with one or two selected vendors would be the stronger position. For a research prototype optimising for cost, portability and speed of iteration, the gateway remains the correct selection.

4. Model selection

4.1 Generation model: Poolside Laguna S 2.1

Table 2. Declared characteristics of the generation model [3].

Property	Value
Identifier	poolside/laguna-s-2.1:free
Vendor	Poolside
Architecture	Mixture of experts, 118 billion total parameters, 8 billion active
Context window	262,144 tokens
Maximum output	32,768 tokens
Tool invocation	Supported
Enforced structured output	Not supported. A response format cannot be specified, so JSON output is not guaranteed at the interface level
Licence	Open weight, OpenMDW-1.1
Reported capability	70.2 percent on Terminal Bench 2.1 and 40.4 percent on DeepSWE
Cost	Provided without charge

Justification. The context window of 262,144 tokens is the decisive property. Prompts issued by AdaptED are long by construction: a curriculum analysis carries entire retrieved chapters, and an outcome analysis carries a course outline of six to seven pages together with the manual vocabulary against which it is checked. A model with a short context window would compel aggressive truncation and would discard precisely the cross references on which the validation rules operate. Support for tool invocation aligns with the action based agent model, the weights are openly licensed so that a third party may reproduce the result, and the endpoint carries no cost.

Constraint arising from this selection. Because a response format cannot be enforced at the interface, JSON output must be secured within the application. AdaptED already implements the correct response: the prompt states the required contract, the reply is parsed and validated against the agent's declared schema, and a validation failure is retried under the bounded retry policy, with the savepoint discarding the writes of the failed attempt. It is recommended that this be presented explicitly as a design response to a documented model limitation rather than left implicit in the code, and that the rate of parsing failure be recorded as a reportable metric.

Second constraint. Input and output submitted to the zero cost endpoint may be retained by the provider and used to improve its models [3]. Student names, identifiers and submitted answers must therefore not be transmitted through that endpoint. Synthetic or anonymised data should be used for any public demonstration.

4.2 Embedding model: NVIDIA Llama Nemotron Embed VL 1B v2

Table 3. Declared characteristics of the embedding model [4][5].

Property	Value
Identifier	nvidia/llama-nemotron-embed-vl-1b-v2:free
Vendor and family	NVIDIA, Nemotron retrieval collection
Type	Dual encoder combining language and vision
Accepted input	Text, or the page image of a document
Output	A single vector of 2,048 dimensions , obtained by mean pooling
Context	32,768 tokens
Licence	NVIDIA Open Model License, approved for commercial use
Cost	Provided without charge

Justification. The material indexed by AdaptED is substantially tabular and diagrammatic, comprising syllabi, mapping matrices, weekly plans and figures. A text only encoder perceives such material only through flattened extracted text, which destroys table structure and therefore loses the relationships that give the content meaning. The selected model is a visual document retriever: a page image may be embedded directly, so that layout is preserved as a retrieval signal. The dual encoder design also matches the shape of the pipeline, since the corpus is embedded once at upload while only the query is embedded at request time.

Principal constraint: dimensionality. The model emits vectors of 2,048 dimensions, while the approximate index types provided by pgvector operate on the standard vector type up to a limit of 2,000 dimensions [6]. The vectors may therefore be stored but cannot be indexed in their native form. Three remedies are available, in descending order of preference:

- Store the column in the half precision vector type and build the index on that. Half precision indexing accommodates a higher dimensional limit and halves the storage requirement, and is the least invasive of the three.
- Reduce the vectors to 2,000 dimensions or fewer before insertion, reporting the reduction method used and its measured cost to retrieval quality.
- Leave the column unindexed and accept exhaustive scanning. This is acceptable at prototype corpus sizes but degrades linearly and will not sustain a realistic course library.

The applicable limit depends on the installed version of the extension and should be confirmed empirically in the target environment before the choice is committed to a written report.

Recorded alternative. The text only model of the same family supports a variable output dimension chosen at inference time, which removes the indexing constraint entirely, and has been evaluated across twenty six languages including Bengali [7]. Should Bangla language material or the indexing limit become material, this constitutes the documented fallback. The cost of that substitution is the loss of page image retrieval.

5. Deterministic offline execution mode

Configuring the provider to its mock setting executes the entire platform with no credential and no network access. This is an architectural decision rather than a development convenience, and rests on four grounds:

- **Reproducibility.** Test outcomes do not shift when a provider revises a model checkpoint, so a result recorded today can be regenerated at a later date.
- **Verifiability by a third party.** The test suite executes without secrets, so the repository can be cloned and verified by an examiner or reviewer who holds no credential.
- **Demonstrability.** An examination or demonstration can proceed over an unreliable network connection.
- **Separation of concerns.** The mode enforces a clean boundary between orchestration logic and model output, and it is this boundary that makes the model substitution described in Section 3.3 possible at all.

The significance of this mode is greatest for the outcome mapping agent. Its compliance checks are deterministic evaluations of rules drawn from the AIUB Computer Science manual and execute entirely offline; the language model is optional and is used only to refine summary prose. A compliance verdict on which a faculty member may act must be reproducible and explainable, and so deriving it from rules rather than from a probabilistic model is a correctness requirement rather than an optimisation. This point merits explicit emphasis in any written account of the system.

6. Risk register

Table 4. Identified risks arising from the selections above, with mitigations.

Risk	Impact	Mitigation
Withdrawal of a zero cost endpoint	The pipeline ceases to function. The risk is demonstrated rather than theoretical: the largest model of the same family has already been withdrawn from the zero cost tier, as have the corresponding Llama and Qwen tiers [8].	No model identifier is to be fixed in application code. Identifiers remain in configuration, the deterministic mode is kept in working order, and the identifier together with the date is recorded against every experimental run.
Rate limiting on the zero cost tier	A single submission initiates six or more model calls, so a modest number of concurrent students can exhaust a daily allowance.	Cache embeddings so that a document is indexed once rather than on every query, batch embedding requests, apply exponential backoff on a rate limit response, and provision paid capacity in advance of any multiple user evaluation.
Structured output is not enforced	A malformed reply interrupts the handover to the next agent in the sequence.	Addressed in the existing design through the stated prompt contract, schema validation and bounded retries with per attempt savepoints. The rate of parsing failure should be logged and reported.
Retention of submitted data for provider training	Student data leaves the institution and may be retained by a third party.	Use synthetic or anonymised data. Should genuine data become necessary, migrate to a paid endpoint and set the routing policy to refuse data collection.
Vector dimensionality exceeds the index limit	Similarity search cannot use an approximate index, so query latency grows with corpus size.	Migrate the column to the half precision type, or reduce dimensionality. The decision should be taken before the corpus grows, since altering a populated column is materially more disruptive.
The gateway constitutes a single point of failure	An outage at the gateway halts all generation, notwithstanding the several vendors behind it.	The deterministic mode preserves demonstrability. Beyond prototype stage, one direct vendor credential should be retained as a secondary path.

7. Configuration reference

Environment configuration, read through pydantic-settings:

```
LLM_PROVIDER=openrouter
OPENROUTER_API_KEY=sk_or_your_key_here
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
LLM_MODEL=poolside/laguna-s-2.1:free
EMBEDDING_MODEL=nvidia/llama-nemotron-embed-v1-1b-v2:free
EMBEDDING_DIM=2048
DATABASE_URL=postgresql+psycopg://user:pass@localhost:5432/adapted
```

Both request types are issued through the same client, credential and base address. Only the endpoint and the model identifier differ:

```
client = OpenAI(base_url=settings.openrouter_base_url,
                api_key=settings.openrouter_api_key)

# generation
chat = client.chat.completions.create(
    model=settings.llm_model, messages=messages)

# retrieval
vec = client.embeddings.create(
    model=settings.embedding_model, input=chunks)
```

For entirely offline operation the provider is set to its mock value and the credential is left unset. Outcome mapping validation is unaffected, since its rules are deterministic.

8. Validity of this document

Model availability, pricing and zero cost status on aggregation gateways change on a weekly cycle. The model characteristics recorded in Tables 2 and 3 were verified against the sources listed below at the time of writing. They should be reverified against the live catalogue before this document is submitted or relied upon.

References

- [1] OpenRouter. Embeddings API reference. https://openrouter.ai/docs/api_reference/embeddings
- [2] OpenRouter. One interface for image, video, audio, embeddings and transcription. <https://openrouter.ai/blog/insights/every-modality-one-api/>
- [3] OpenRouter. Laguna S 2.1 (free): pricing, providers and specifications. <https://openrouter.ai/poolside/laguna-s-2.1:free>
- [4] NVIDIA. Model card, llama-nemotron-embed-v1-1b-v2. <https://huggingface.co/nvidia/llama-nemotron-embed-v1-1b-v2>
- [5] OpenRouter. Text embedding model collection. <https://openrouter.ai/collections/embedding-models>
- [6] pgvector. Extension documentation, index dimensionality limits. <https://github.com/pgvector/pgvector>
- [7] NVIDIA. Model overview, llama-nemotron-embed-1b-v2. <https://docs.api.nvidia.com/nim/reference/nvidia-llama-nemotron-embed-1b-v2>
- [8] OpenRouter. Free model collection. <https://openrouter.ai/collections/free-models>

End of document.